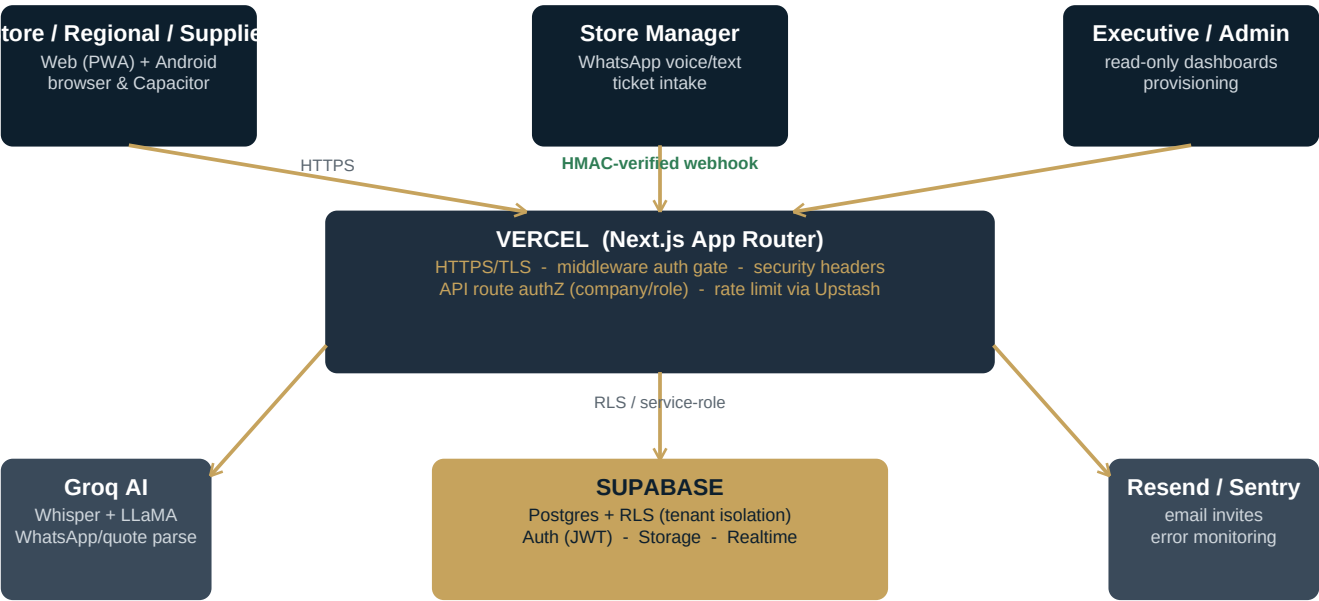


Why the stack - service purposes, security roles & how they fit

Motiv is a mobile-first maintenance ticketing and quoting platform. Its stack is deliberately split so each provider does one job well and - critically - so that **security controls sit at defined trust boundaries**. This document explains what each service is for, how they connect, and where each one protects data and information flow.

Everything below currently runs on **free / hobby tiers** (dev). See `docs/INFRASTRUCTURE_TIERS.md` for tier limits and what unlocks on paid.

How it fits together (request & trust boundaries)



Flow: a client (web PWA, Android wrapper, or a WhatsApp message) reaches the app only over HTTPS. **Vercel** is the gateway - it terminates TLS, runs the middleware auth gate, sets security headers, and every API route re-checks the caller's company + role before touching data. **Supabase** holds all state; its Row-Level Security is the last line of tenant isolation. Side services (Groq, Resend, Sentry) are called server-side only, never directly by the browser.

At a glance

Service	Purpose in Motiv	Security / safety role
Supabase	Postgres database, Auth, file Storage, Realtime. The single source of truth.	RLS multi-tenant isolation; JWT auth; service-role kept server-side only.
Vercel	Hosts the Next.js app + serverless API routes + cron jobs.	HTTPS/TLS, edge middleware auth gate, security headers, secret storage.
Upstash	Redis - shared counter for rate limiting across serverless instances.	Abuse / brute-force / DoS throttling that actually works fleet-wide.
Groq	Whisper (speech-to-text) + LLaMA (extraction) for WhatsApp intake & quote-PDF parsing.	Server-side only; PII (voice/text) leaves our boundary here - a privacy consideration.
WhatsApp Cloud API	Low-friction ticket intake channel for store managers (voice/text).	Inbound webhook is HMAC-verified (x-hub-signature-256); sender matched to a registered user.

Resend	Transactional email - invites, onboarding, password delivery.	Account-provisioning channel; controls who can create an account.
Sentry	Error & performance monitoring across API, client and cron.	Detect incidents/attacks fast (MTTD); no blind spots in production.
Web Push / VAPID	Browser/PWA push notifications for ticket events.	VAPID-signed, per-user subscription; no third-party message broker.
Capacitor (Android)	Thin native wrapper pointing at the live site.	Cleartext disabled, HTTPS-only WebView; release signed with our keystore.

Service deep-dive

Supabase - the secure core

Purpose. Postgres database, authentication, object storage and realtime, all in one. Every ticket, quote, completion, rating and profile lives here.

Security & safety. This is where multi-tenancy is enforced. **Row-Level Security (RLS)** policies scope almost every table to `company_id = app_company_id()` plus role/link checks, so one company can never read another's data at the database layer. Auth issues short-lived JWT sessions. The powerful **service-role key** (which bypasses RLS) is used only in server code for cross-user fan-out (e.g. notifications) and never shipped to the browser.

How it connects. Vercel's server code talks to Supabase two ways: a *user-scoped* client (RLS-enforced, for the logged-in user) and an *admin* client (service-role, for controlled cross-user operations). Storage holds photos, COC certificates and quote PDFs.

Vercel - the gateway & enforcement point

Purpose. Hosts the Next.js App Router site, the serverless API routes, and scheduled cron jobs. Every request enters here.

Security & safety. Vercel is the outer trust boundary: it forces HTTPS/TLS, runs **middleware** as the single gate for page-route access, and (added in hardening) emits security headers (HSTS, X-Frame-Options, nosniff, CSP). API routes perform application-level authorization - checking the caller's company and role before mutating - which is the real guard whenever the admin client is used. Secrets (service-role key, API keys) live in Vercel env vars, never in the bundle.

How it connects. Sits between the clients and everything else. Calls Supabase, Groq, Resend and Sentry server-side; the browser only ever talks to Vercel.

Upstash (Redis) - abuse protection

Purpose. A shared, low-latency Redis used as the backing store for rate limiting.

Security & safety. On Vercel's serverless fleet, an in-memory counter is per-instance, so limits are trivially bypassed by spreading requests. Upstash gives a **global** counter, so write endpoints (quotes, provisioning, onboarding, the expensive AI parse route) are throttled fleet-wide - protecting against brute-force, scraping and cost-blowout / DoS.

How it connects. Called from the rate-limit helper on API routes; falls back to in-memory if the Upstash env vars are absent (dev).

Groq - the AI intake engine

Purpose. Runs Whisper (speech-to-text) and LLaMA (structured extraction) to turn a WhatsApp voice note or a messy quote PDF into clean ticket / quote fields.

Security & safety. Called only from server routes, never the browser. Note the **information-flow consideration**: the content sent for transcription/extraction (voice notes, message text, quote documents) leaves Motiv's boundary and is processed by a third party - relevant to POPIA and to what we tell users in the privacy policy. Amounts are still parsed deterministically in our own code, not trusted from the model.

How it connects. Vercel → Groq (OpenAI-compatible API). Output is clamped to the exact ticket enums before anything is written to Supabase.

WhatsApp Cloud API - the intake channel

Purpose. Lets a store manager log a ticket by sending a WhatsApp voice note or text - the lowest-friction channel for the field.

Security & safety. The inbound webhook is a public URL, so it is **HMAC-verified** (x-hub-signature-256 against the Meta app secret) before any processing, and the sender's phone number must match a registered user. This stops attackers from forging ticket/photo events.

How it connects. Meta → Vercel webhook → Groq (parse) → Supabase (draft session, then ticket). Confirmation flows back to the user over WhatsApp.

Resend, Sentry, Web Push, Capacitor - supporting roles

Purpose. Resend sends transactional email (invites, onboarding, passwords). Sentry captures errors. Web Push/VAPID delivers notifications. Capacitor wraps the site as an Android app.

Security & safety. Resend is effectively an **account-provisioning gate** (who receives an invite/credential). Sentry is a **safety net** - fast detection of failures or abuse in production. Web Push is VAPID-signed per user. The Android wrapper disables cleartext and loads the site over HTTPS only, signed with our private keystore.

How it connects. All are invoked server-side from Vercel (except the Android wrapper, which is the client shell); each no-ops cleanly if its keys are unset.

Security summary - where data is protected along the path

- **In transit:** HTTPS/TLS everywhere (Vercel + Android WebView cleartext disabled).
- **At the edge:** middleware auth gate + security headers on every response.
- **At the API:** per-request company + role authorization before any admin-client write.
- **At the database:** Supabase RLS isolates tenants; service-role key stays server-side.
- **Against abuse:** Upstash-backed rate limiting on write/expensive routes.
- **At untrusted inputs:** HMAC-verified WhatsApp webhook; deterministic amount parsing.
- **Known gaps (in progress):** storage buckets are still public + served via permanent URLs (moving to private + signed URLs); privacy disclosure for AI processing of PII via Groq.